School of CSE, REVA University

Codebase-Wide Version Migration & Refactoring Agent

System Design Document

Team: Adithya Naik (R23EF014), Prajwal Ragavendra Hawaldar (R23EF183), Verril Vaz (R23EN179), Nandan P (R23EH082), Prabanjan R (R23EF181)
Faculty Guide: Dr. Nimrita Koul, Center for Artificial Intelligence & Machine Learning
Problem Statement Identifier: CAIML_Nimrita_Koul_3_PS_209
September 2026

1. Introduction

1.1 Purpose

This document specifies the design of the Codebase-Wide Version Migration and Refactoring Agent. It describes the system architecture, the design of each module, the structure of the data the modules share, the interfaces between modules and with external tools, and the dynamic behaviour of the system during a migration run. It is written for the development team and for the faculty guide, and it follows on from the literature survey and requirement analysis submitted earlier, from which the functional requirements referenced in Section 11 are taken.

1.2 Scope

The design covers the nine modules identified for the project: specification and configuration, static analysis and call-graph mapping, planning and batching, graph-based memory, the edit engine, sandboxed verification, failure analysis and self-correction, trajectory logging and patch generation, and the benchmark and evaluation harness. The primary migration used to validate the design is the upgrade of database access code from SQLAlchemy 1.4 to 2.0, with Python 2 to 3 and Python 3.8 to 3.12 as secondary cases.

1.3 Definitions and Abbreviations

Term	Meaning
AST / CST	Abstract syntax tree / concrete syntax tree, used to parse source files
SCC	Strongly connected component, a set of files with a circular dependency
DFD	Data flow diagram
LLM	Large language model, used to propose patches and targeted fixes
Call site	A location in the code that invokes a deprecated API
Impact set	The call sites and their transitive callers affected by a migration
Batch	A group of files edited and verified together in one iteration
Oracle detector	An independent checker that counts deprecated usages, used only to measure completeness

1.4 References

[1] Literature Survey and Requirement Analysis, CAIML_Nimrita_Koul_3_PS_209, Task 1. [2] Team Overview, Module Description and Technology Stack, Internship Week 1.

2. System Overview

The system takes a migration specification and a target Python repository as input and produces a unified patch that applies the migration, together with a structured log of every decision made while producing it. Three design principles guide the system.

Deterministic work first. Mechanical rewrites are performed by codemods and are not left to the language model. The model is used only where a call site needs judgement or where a test failure must be diagnosed, which improves reproducibility and reduces cost.

Shared memory over conversation history. The state of every file, together with pending fixes and short summaries, is kept in a graph memory outside the model prompt. Each model call is built from a bounded slice of this graph rather than from the full run history.

Verify before trusting an edit. No batch is considered complete until the linter and the relevant tests pass inside an isolated sandbox. A failure is traced back through the call graph to the change that caused it before a repair is attempted.

3. Design Goals and Constraints

Goal	Design response
Reproducibility	Fixed model temperature, pinned dependency versions, every prompt and tool result logged with a step identifier
Isolation	All test and lint execution occurs in a Docker container with resource and network limits; the working repository is only changed through Git commits
Token and step efficiency	Context assembled per node from the graph memory under a configured budget, instead of the full conversation history
Recoverability	Git checkpoint after every batch; a batch that cannot be repaired within a retry limit is rolled back rather than left partially edited
Extensibility	Each module exposes a fixed interface (Section 5) so that, for example, the parser or the sandbox can be replaced without changing the other modules

Table 1. Design goals and how the architecture satisfies them

4. System Architecture

4.1 Architectural Style

The system combines a pipeline architecture with a blackboard architecture. Modules 1, 2 and 3 execute once per run in a fixed order to prepare a plan. Modules 5, 6 and 7 then execute repeatedly in a verify-and-repair cycle for each batch until the batch passes or its retry limit is reached. Module 4, the graph memory, acts as the blackboard: every other module reads and writes to it instead of passing its full output directly to the next module in the pipeline. This keeps each stage decoupled and keeps the language model's context bounded no matter how many batches the run contains.

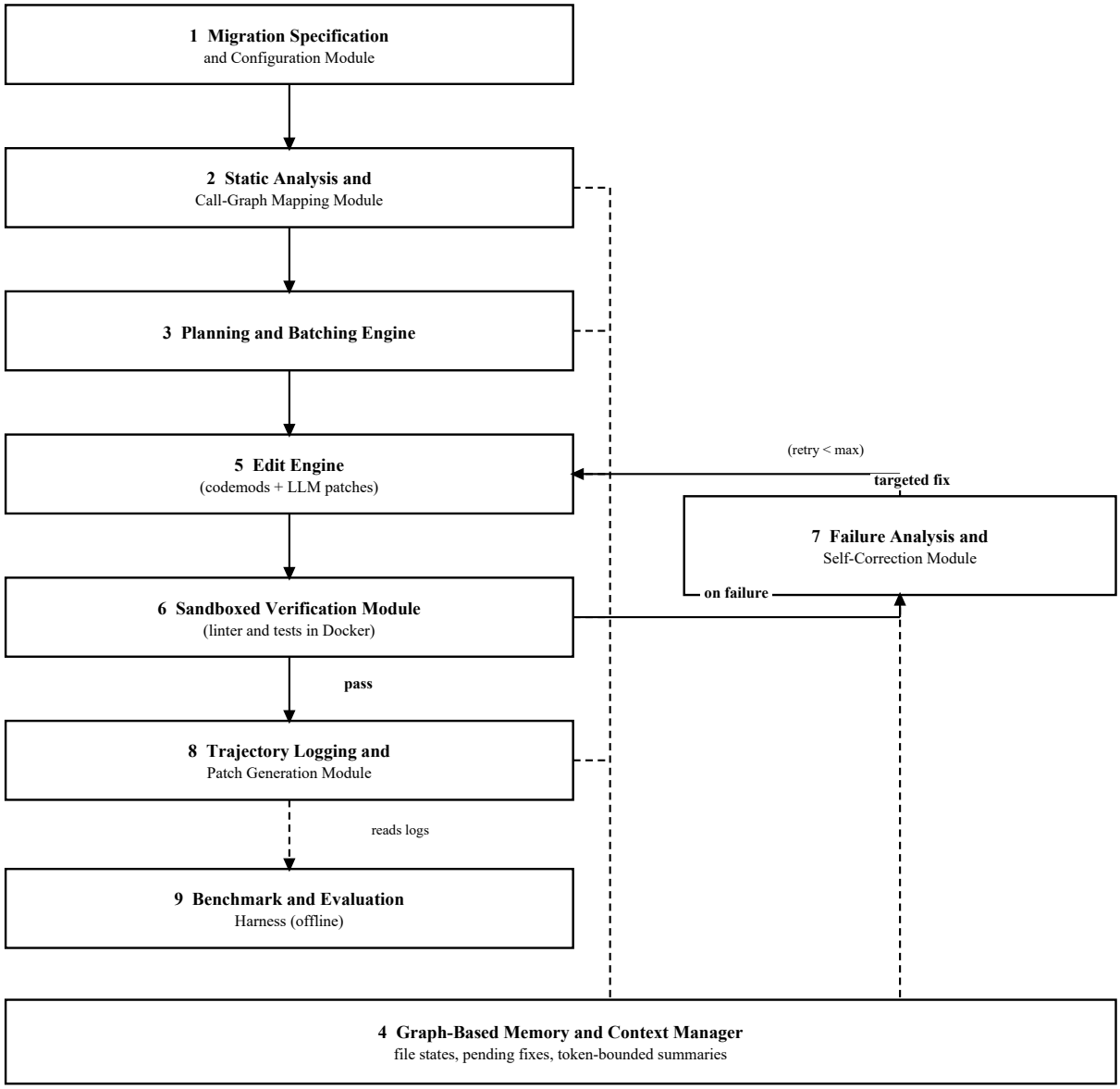


Fig. 1. System architecture. Solid arrows show the main pipeline; the loop on the right is the verify-and-repair cycle between the Edit Engine and the Failure Analysis module; dashed lines show every module's access to the shared graph memory.

5. Module (Component) Design

Table 2 specifies, for every module, the operations it provides to the rest of the system and the components it requires in turn. The full responsibility of each module was described in the Task 2 submission; this table states the design contract used to implement it.

ID / Name	Provided interface	Required interface	Consumes
1. Migration Specification and Configuration Module	validate(spec_file), load_config(run_args)	MigrationSpec object, RunConfig object	Specification file, CLI arguments
2. Static Analysis and Call-Graph Mapping Module	analyze(repo_path, spec) -> AnalysisResult	ast / LibCST / tree-sitter parser, file system read access	Repository source files, MigrationSpec

ID / Name	Provided interface	Required interface	Consumes
3. Planning and Batching Engine	plan(dependency_graph, impact_set) -> BatchPlan	Dependency graph and impact set from Module 2	Dependency graph, impact set, configuration limits
4. Graph-Based Memory and Context Manager	get_state(file), update_state(file, state), build_context(node, budget)	Persistent store (JSON or SQLite), token counter	Graph, plan, edit and verification results
5. Edit Engine	apply_codemod(rule, file), propose_patch(context) -> Patch, validate(patch)	LibCST codemod library, LLM API client, Git	Batch, call-site data, context from Module 4
6. Sandboxed Verification Module	run_lint(files), run_tests(selection) -> Report	Docker engine, pytest, Ruff, Git checkpoint	Edited repository state, test selection
7. Failure Analysis and Self-Correction Module	diagnose(report) -> FixRequest, is_oscillating(history)	Call graph, graph memory, patch history	Test and linter reports, stack traces
8. Trajectory Logging and Patch Generation Module	log_event(entry), emit_patch(repo) -> patch_file	Versioned JSON schema, Git diff	Events from all modules, final repository state
9. Benchmark and Evaluation Harness	run_suite(tasks, config) -> MetricsReport	Task corpus, ablation configuration	Benchmark tasks, logs from Module 8

Table 2. Module interface summary

5.1 Notes on the Static Analysis Module

Module 2 resolves aliases and re-exports through a symbol table before matching call sites against the patterns named in the migration specification, so that a call reached through *from sqlalchemy import query as q* is still found. Sites that depend on dynamic dispatch, such as getattr or eval, cannot be resolved statically and are reported to Module 4 as unresolved rather than silently skipped.

5.2 Notes on the Graph-Based Memory Module

The context assembled for a model call always contains the target file, its direct neighbours in the dependency graph, any open failures linked to it, and a short summary of already completed work, truncated to the configured token budget. This is the mechanism that lets the system operate over repositories larger than a single prompt.

5.3 Notes on the Failure Analysis Module

A stack trace is parsed into frames, each frame is mapped to a file and function in the dependency graph, and the graph is walked upward from the failing frame to the nearest edited node, which is treated as the likely cause. The module also compares each new patch against the file's patch history so that a repair that repeats an earlier, already-rejected change is detected and escalated instead of retried indefinitely.

6. Data Design

6.1 Graph Memory Schema

Field	Type	Description
path	string	Repository-relative file path; the node identifier
state	enum	pending, analysed, edited, verified, failed, or rolled_back
content_hash	string	Hash of the file's current content, used to detect drift
dependencies	list[string]	Files this file imports from or calls into
dependents	list[string]	Files that import from or call into this file

Field	Type	Description
call_sites	list[CallSite]	Breaking call sites found in this file by Module 2
pending_fixes	list[string]	Fix identifiers awaiting application
summary	string	Condensed note of completed work, kept under a token limit
retry_count	integer	Number of repair attempts made on this file

Table 3. FileNode record (one per source file, keyed by path)

A CallSite record carries file, line, column, enclosing_function, matched_pattern, and a status of pending, patched, or verified. A dependency edge carries source, target, and a type of import, call, or inherit, together with the symbol name that creates the dependency.

6.2 Migration Specification Schema

A specification carries an identifier, a target language or library version, a list of deprecated patterns (each with an AST matcher and a description), a list of replacement rules (each naming either a codemod or a prompt template), and a reference to the oracle detector used to score completeness for that migration.

6.3 Trajectory Log Schema

Each log entry carries a step identifier, a timestamp, the module that produced it, the action taken, the batch and files affected, the tokens used, and a result status. This is the schema written by Module 8 and read by Module 9 when computing benchmark metrics.

7. Interface Design

7.1 Command-Line Interface

Command	Effect
migrate plan --spec FILE --repo PATH --dry-run	Runs Modules 1 to 3 and prints the call-graph mapping and the batch plan without editing any file
migrate run --spec FILE --repo PATH [--resume]	Executes the full migration loop; resumes from the last checkpoint in the graph memory if given --resume
migrate status --repo PATH	Prints the current state of every file in the graph memory
migrate evaluate --tasks DIR --config FILE	Runs Module 9 over a set of benchmark tasks and reports aggregate metrics

Table 4. Command-line interface

7.2 External Interfaces

LLM API. The edit engine and the failure analysis module send a request containing a system prompt, a bounded context and a task instruction at temperature zero, and receive a unified diff or a fix explanation in return. **Sandbox execution.** The verification module sends a command, a working directory and a timeout to the Docker engine with network access disabled, and receives standard output, standard error and an exit code. **Version control.** The edit engine and the verification module checkpoint, diff and roll back through Git; no module writes to the repository outside a Git-tracked change.

8. Dynamic Behaviour

8.1 Data Flow

Figure 2 gives the context-level view of the system as a single process exchanging data with the developer, the repository and the external language model. Figure 3 decomposes that process into the six processes that correspond to Modules 2, 3, 5, 6 and 7, plus a logging process for Module 8, and shows their shared use of the graph memory data store.

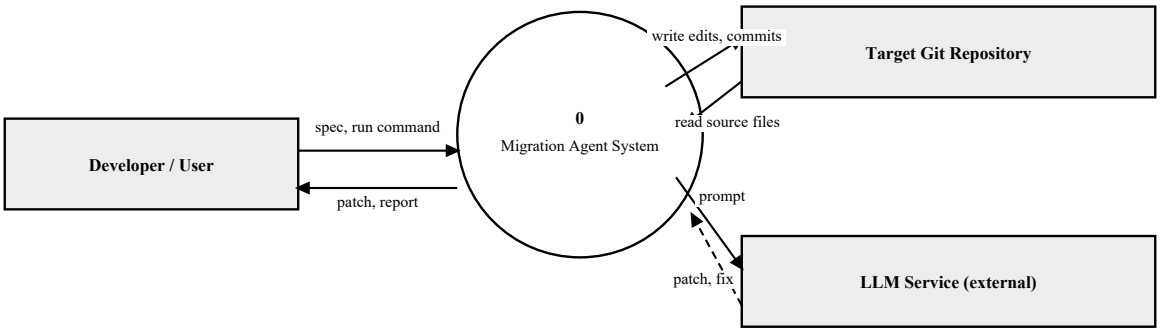


Fig. 2. Data flow diagram, level 0 (context diagram).

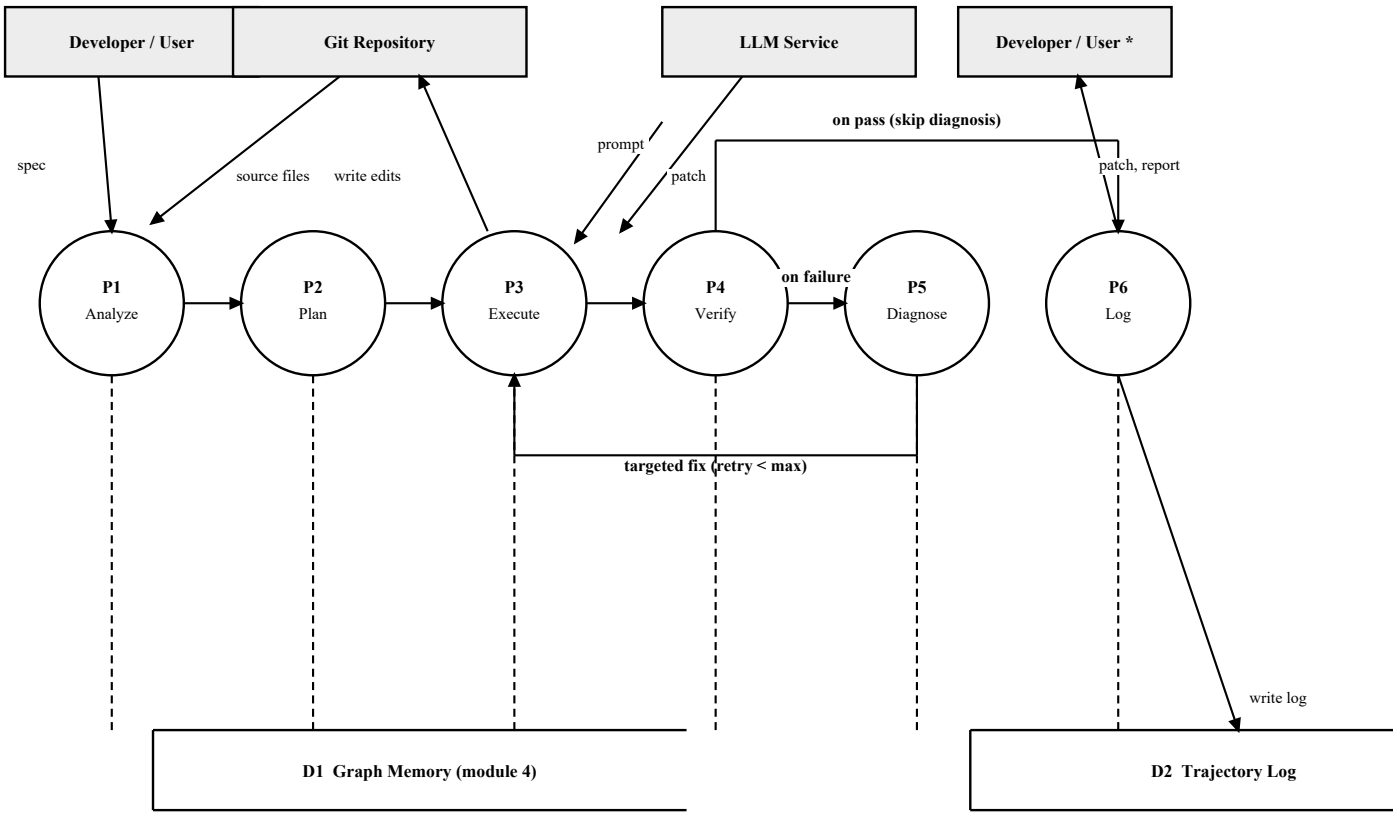


Fig. 3. Data flow diagram, level 1. P1 to P6 correspond to Modules 2, 3, 5, 6, 7 and 8; D1 is the graph memory of Module 4 and D2 is the trajectory log. The asterisk marks a second occurrence of the Developer / User entity, repeated so that its two flows do not cross the diagram.

8.2 Batch Execution Sequence

Figure 4 shows the message sequence for one batch. The Planner dispatches a batch to the Executor, which reads a bounded context from Memory, obtains a patch, and hands the batch to the Verifier. The alt fragment shows the two outcomes: on success the batch is marked verified and control returns to the Planner; on failure the Diagnoser locates the breaking node in Memory and returns a targeted fix to the Executor, which repeats the cycle up to the configured retry limit.

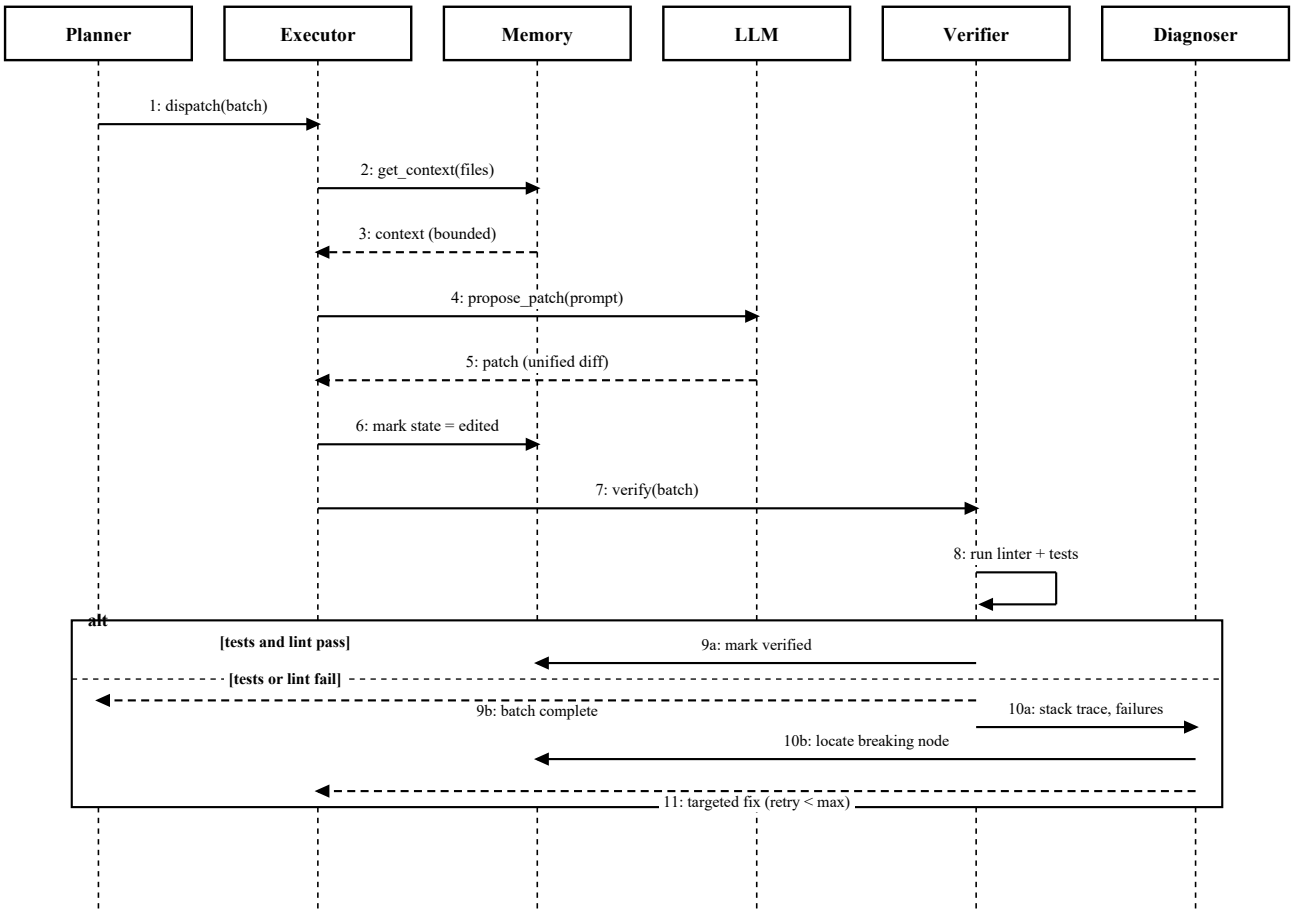


Fig. 4. Sequence diagram for the batch execution and verification cycle.

8.3 File State Lifecycle

Figure 5 shows the states a file passes through, matching the state field of the FileNode record in Section 6.1. Verified and Rolled Back are the two terminal states of a run.

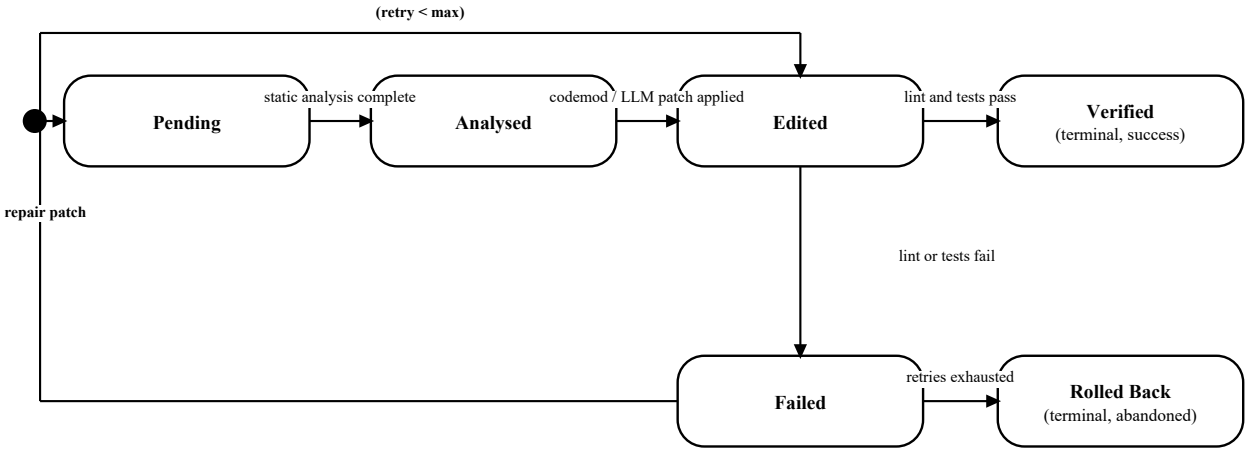


Fig. 5. State diagram for the migration state of a single file.

9. Deployment Design

The agent process runs on the host machine and is the only component with write access to the Git working copy. It never runs tests directly against that copy; instead it asks the Docker engine to execute the linter and the test suite against an isolated copy of the repository inside a sandbox container with network access disabled, which satisfies the isolation goal in Section 3. The only network call the agent process makes is to the external LLM API over HTTPS, to request patches and targeted fixes.

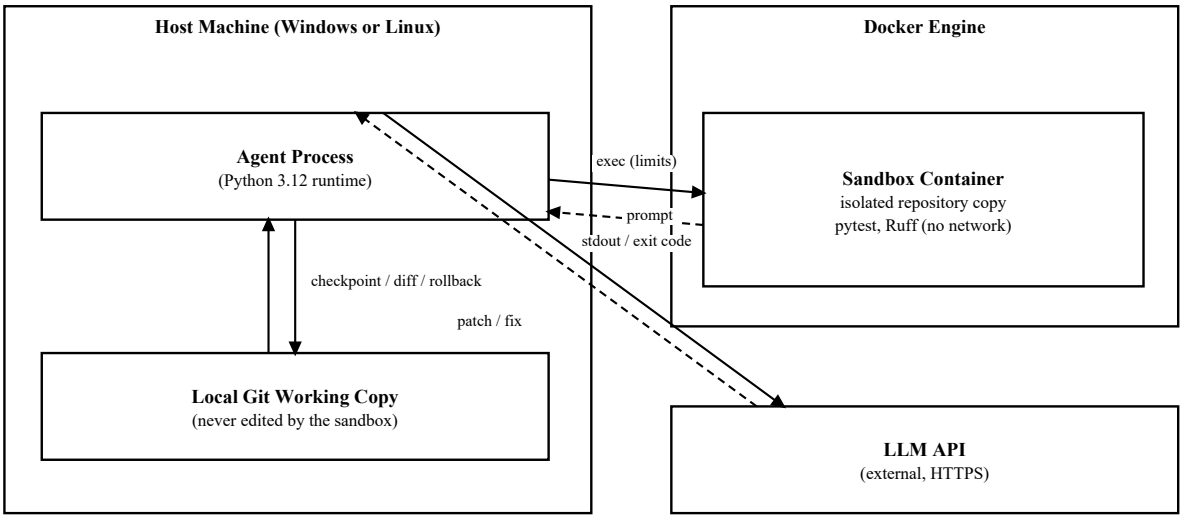


Fig. 6. Deployment diagram showing the host process, the isolated Docker sandbox, and the external language model service.

10. Design Constraints and Assumptions

Constraint or assumption	Consequence for the design
The repository parses as valid Python 3 source	Python 2 sources are converted with a fallback grammar before Module 2 runs
Dynamic dispatch defeats static resolution	Module 2 reports unresolved sites; the verification loop is the fallback safeguard for them
The test suite is mostly deterministic	A baseline run before migration records already-failing tests so that regressions can be told apart from them
The LLM API has request and rate limits	Modules 5 and 7 batch and cache requests, and a run stops cleanly at the configured token or step ceiling
Sandbox execution has no network access	Package installation is completed once before the sandbox image is sealed

Table 5. Design constraints and assumptions

11. Requirements Traceability

Table 6 maps each module to the functional requirements it implements, using the requirement identifiers defined in the Task 1 requirement analysis, so that every requirement in that document is covered by exactly one module.

Module	Functional requirements covered
1. Migration Specification and Configuration	FR1, FR15, FR16
2. Static Analysis and Call-Graph Mapping	FR2, FR3, FR4, FR5
3. Planning and Batching Engine	FR6
4. Graph-Based Memory and Context Manager	FR7, FR8
5. Edit Engine	FR9
6. Sandboxed Verification Module	FR10
7. Failure Analysis and Self-Correction	FR11, FR12
8. Trajectory Logging and Patch Generation	FR13
9. Benchmark and Evaluation Harness	FR14

Table 6. Traceability from modules to functional requirements FR1 to FR16

12. Conclusion

This document has specified the architecture, module interfaces, data schema, control flow and deployment of the migration agent, and has traced every module back to the functional requirements it satisfies. Implementation will proceed module by module in the order given in the project plan, starting with the static analysis module, since every later module depends on the call-graph mapping it produces.